

Cloud, DevOps and Security 101

2025 - Bruynseraede Quinten

Welcome!

 Quinten

 Data & Cloud Engineer

 D'leteren, Telenet, KBC

 AWS / Terraform / Platform Automation / go lang

 Tech Lead D&C + Competence Lead Devops & SWE

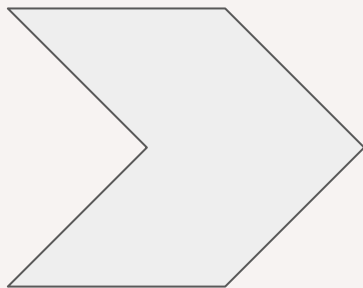


Today

- High-level introduction to some terminology you need to know
- Mainly theory, small exercises

Today

PRINCIPLES



TOOLS

What is Cloud?

What is Cloud?

- **Computing resources you “rent” from a Cloud provider**
- Compare it to electricity in your home
- Other names: *something* as a Service
- IaaS, PaaS, SaaS, ...
- Lots of software today is distributed as a “cloud” version

What is Cloud?

1. **ON DEMAND**
2. **PAY AS YOU GO**
3. **AVAILABLE OVER THE INTERNET (API DRIVEN)**

What is Cloud?

On Premise	IaaS	PaaS	FaaS	SaaS
Functions	Functions	Functions	Functions	Functions
Application	Application	Application	Application	Application
Runtime	Runtime	Runtime	Runtime	Runtime
Operating system	Operating system	Operating system	Operating system	Operating system
Virtualisation	Virtualisation	Virtualisation	Virtualisation	Virtualisation
Networking	Networking	Networking	Networking	Networking
Storage	Storage	Storage	Storage	Storage
Hardware	Hardware	Hardware	Hardware	Hardware

Legend

- Provider managed
- Customer managed
- On Premise
- Cloud Computing

Different forms of Cloud

- Public Cloud
 - The traditional form
 - You use resources from a cloud provider in an environment that is shared with other people
- Private Cloud
 - Your resources are running in an environment that is isolated from other people
- Hybrid Cloud
 - You use a mix of public and private cloud resources
- Multi Cloud
 - You use different cloud providers at the same time
 - Or on-prem and cloud

Shared responsibility model in public cloud

Cloud provider: security **of** the cloud

You: security **in** the cloud

The big 3 of Cloud

The big 3 of Cloud

- **AWS**
 - Amazon Web Services
 - The “first” cloud provider
 - Largest worldwide
- **Azure**
 - From Microsoft
 - Largest in BE
 - Focus on hybrid cloud
- **GCP**
 - Google Cloud Platform
 - Modern, AI&ML
 - Aggressive marketing

dataroots
experts in data science

Figure 1: Magic Quadrant for Strategic Cloud Platform Services



The others



EU Cloud

HETZNER

 **OVHcloud**

 **Scaleway**

dataroots
experts in data science

	AWS	Microsoft Azure	Google cloud
Caching	Elastic Cache	Redis Cache	Cloud CDN
Processor	EC2	Virtual Machines	Compute Engine
Marketplace	AWS marketplace	Azure Marketplace	G suite Marketplace
GIT Repositories	CodeCommit	DevOps	Cloud source.
Platform as service	Elastic Beanstalk	Cloud Services	App Engine
Storage of Object	S3	Block Blob	Cloud Storage
Managed data warehouse	Redshift	SQL warehouse	BigQuery
Kubernetes Management	EKS	Kubernetes service	Kubernetes engine
File Storage	EFS	Azure Files	ZFS and Avere
Serverless computing	Lambda	Functions	Cloud functions
API management	API Gateway	Azure API Manager	Cloud endpoints
Media services	Elastic Transcoder	Azure media services	Cloud video intelligence API

	AWS	Microsoft Azure	Google cloud
Caching	Elastic Cache	Redis Cache	Cloud CDN
Processor	EC2	Virtual Machines	Compute Engine
Marketplace	AWS marketplace	Azure Marketplace	G suite Marketplace
GIT Repositories	CodeCommit	DevOps	Cloud source
Platform as service	Elastic Beanstalk	Cloud Services	App Engine
Storage of Object	S3	Block Blob	Cloud Storage
Managed data warehouse	Redshift	SQL warehouse	BigQuery
Kubernetes Management	EKS	Kubernetes service	Kubernetes engine
File Storage	EFS	Azure Files	ZFS and Avere
Serverless computing	Lambda	Functions	Cloud functions
API management	API Gateway	Azure API Manager	Cloud endpoints
Media services	Elastic Transcoder	Azure media services	Cloud video intelligence API

Setup your own, free, account

- AWS: <https://aws.amazon.com/free/>
- GCP: <https://cloud.google.com/free>
- Azure: <https://azure.microsoft.com/en-us/free/>

Be careful, free trials are limited!

During labs, dataroots will provide accounts so you don't have to worry about this.

How to interact with “the cloud”

- <https://portal.azure.com/#home>
- CLI tools:
 - `$ az login`
 - `$ az group create`
 - `$ az sql server create -name="my-db" ...`
- **IaC**

How to interact with “the cloud” - IaC

Infrastructure-As-Code lets you write “code” to create cloud resources

Y ec2.yml ×

Users > quinten.bruynseraede > Y ec2.yml > {} Resources

Visualize with Application Composer

```
1 Resources:
1   MyEC2Instance:
2     Type: AWS::EC2::Instance
3     Properties:
4       InstanceType: t2.micro # Change to desired instance type
5       KeyName: my-key-pair
```

How to interact with “the cloud” - IaC

Infrastructure-As-Code lets you write “code” to create cloud resources

 **ec2.tf** 1 ✕

Users > quinten.bruynseraede >  ec2.tf > ...

```
3  resource "aws_instance" "test" {  
2  |    type = "t2.micro"  
1  | }  
4
```

How to interact with “the cloud” - IaC

There are many IaC flavours:

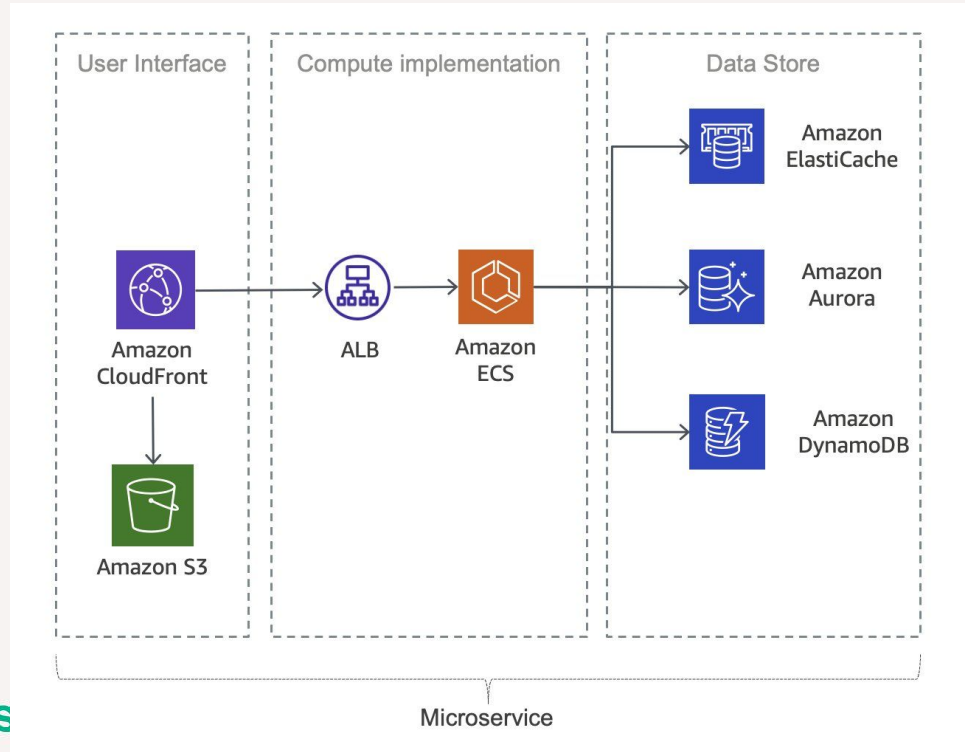
- Terraform (does it all)
- Pulumi / CDK (lets you use familiar programming languages)
- Specific to a cloud provider (CFN, ARM/Bicep, ...)
- Ansible (used for server configuration)
- ...

Why IaC?

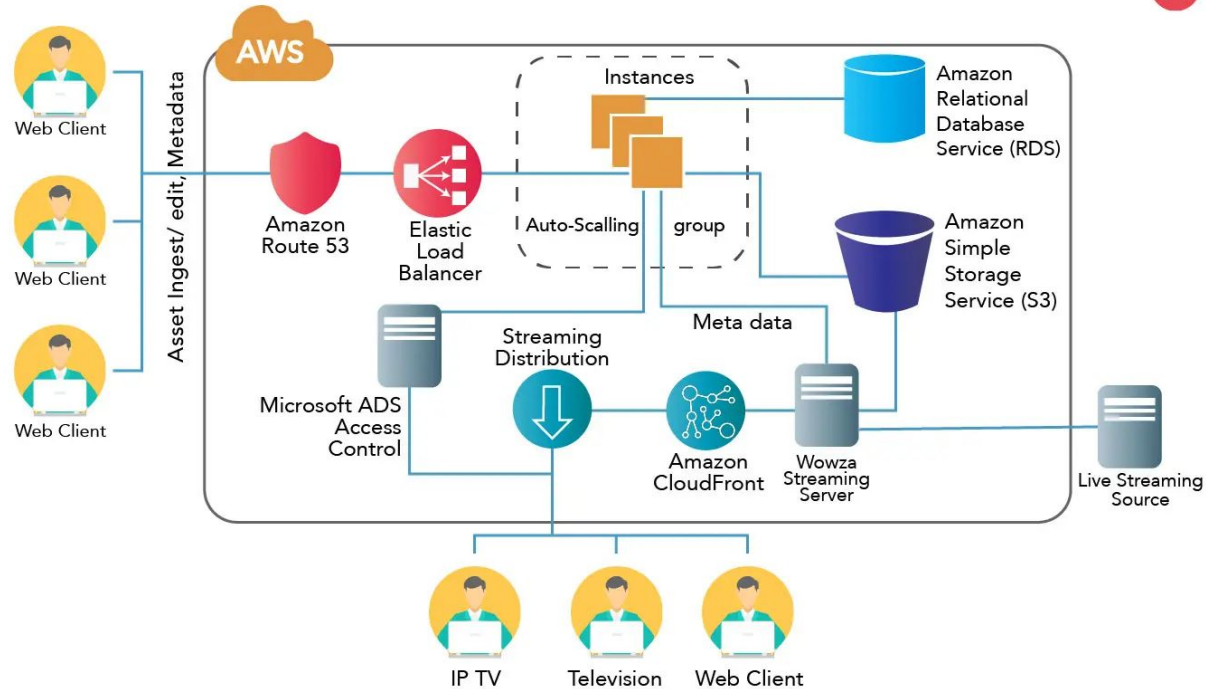
- Reproducible
- Manages full lifecycle of resources: create - update - destroy
- Everything is stored in git
 - Review
 - Rollback
 - Check the context of older changes
 - ...

IaC demo (Terraform)

Cloud architecture

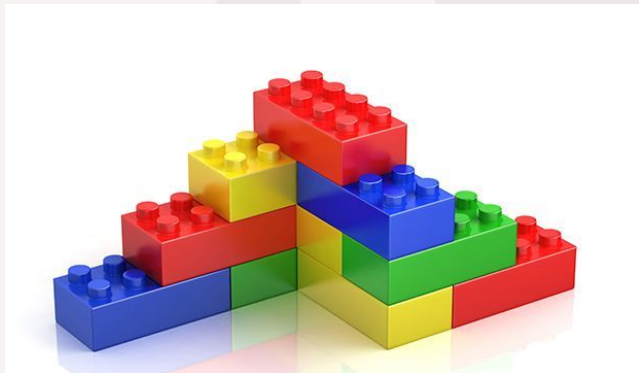


Cloud architecture



Cloud exercise

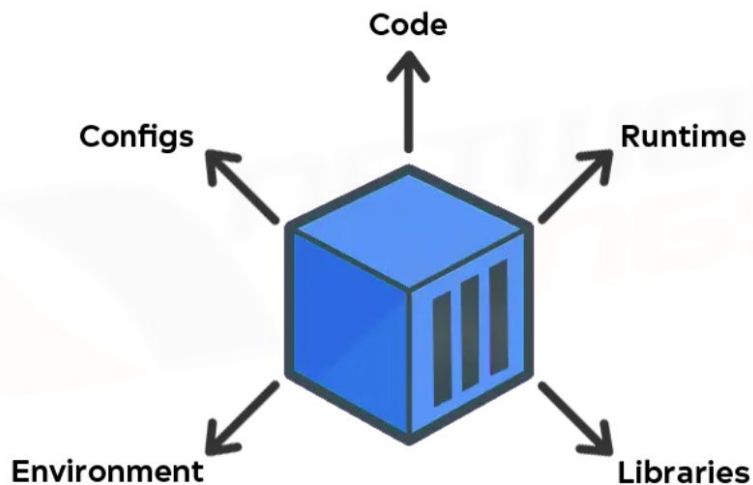
- Design (draw.io) a simple cloud architecture
- Requirements
 - We want a REST API to upload files
 - The uploaded files land on a storage location
 - Every day, a job counts the files and stores the count in a database
- Architecture is about tradeoffs!



Containerization (Docker)

What is a container?

What's included in a Container?

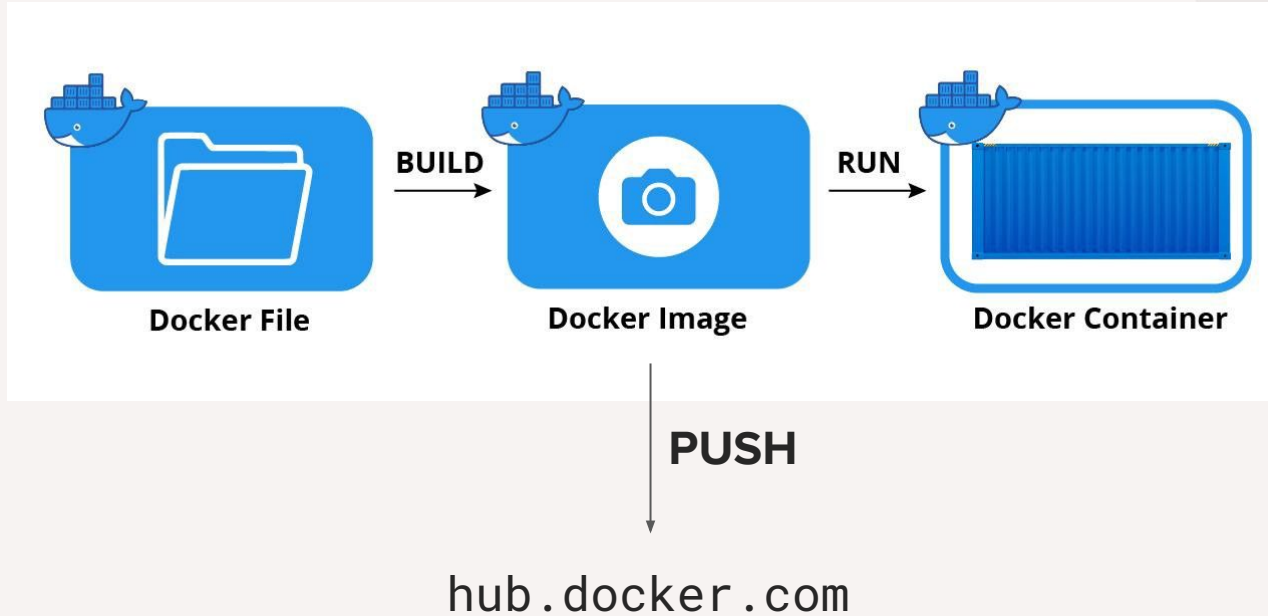


www.nwkings.com

What is Docker?

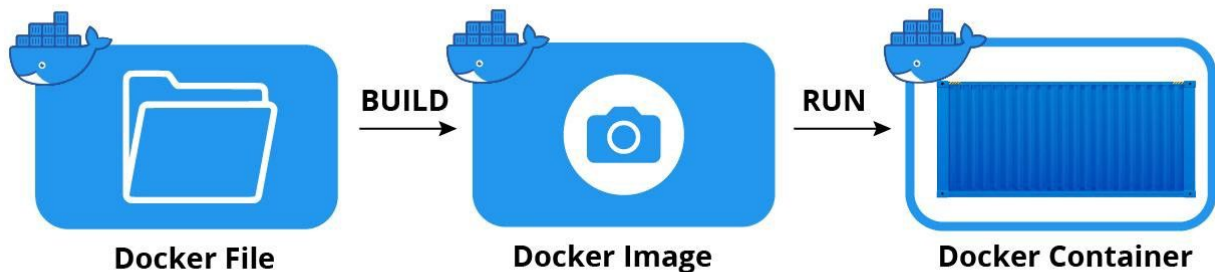
- Open-source project
- Run, build, distribute containers in a standard format
- Started the container revolution
- Also hosts a large public container registry: <https://hub.docker.com/>

How to build a Docker container?



How to build a Docker container?

- `docker build -f Dockerfile` => `docker image`
- `docker push <image>` => `distribute`
- `docker run <image>` => `execute`



How to build a Docker container?



Dockerfile ×

Users > quinten.bruynseraede > Dockerfile > RUN

```
7 FROM klakegg/hugo:0.61.0-ext
6
5 RUN mkdir /drum
4 COPY . /drum
3 WORKDIR /drum
2 ENV HUGO_DESTINATION=/drum_public
1 RUN hugo --baseURL https://mendeddrum.dataroots.io/
8 RUN ls $HUGO_DESTINATION
```

FROM

RUN

COPY

WORKDIR

ENV

How to build a Docker container?

 Dockerfile ✕

Users > quinten.bruynseraede >  Dockerfile >  RUN

```
7 FROM klakegg/hugo:0.61.0-ext
6
5 RUN mkdir /drum
4 COPY . /drum
3 WORKDIR /drum
2 ENV HUGO_DESTINATION=/drum_public
1 RUN hugo --baseURL https://mendeddrum.dataroots.io/
8 RUN ls $HUGO_DESTINATION
```

How to build a Docker container?

 Dockerfile ×

Users > quinten.bruynseraede >  Dockerfile >  RUN

```
7 FROM klakegg/hugo:0.61.0-ext
6
5 RUN mkdir /drum
4 COPY . /drum
3 WORKDIR /drum
2 ENV HUGO_DESTINATION=/drum_public
1 RUN hugo --baseURL https://mendeddrum.dataroots.io/
8 RUN ls $HUGO_DESTINATION
```

Change line 8: only 1 layer is rebuilt!

Docker exercise

1. Appetizer

- Use the Docker CLI to pull a **python** Docker image from Dockerhub
- Run the container. What happens?
- Open an interactive shell in the container and print the environment variables

2. Dockerfile

- Clone <https://github.com/datarootsio/rootsacademy-2024-docker-101>
- Write a dockerfile that:
 - Uses the official Python image as base
 - Copies `main.py` and `requirements.txt` into the image
 - Runs `pip install` in the image to install the required packages
 - Execute `main.py` with Python when you run the image



**Are Docker contains the way to deploy reliable
and scalable applications?**

Container orchestration

Scaling out with containers

How do you add scalability and redundancy to your containers?

Scaling out with containers

How do you add scalability and redundancy to your containers?

How do you handle updating your containers?

Scaling out with containers

How do you add scalability and redundancy to your containers?

How do you handle updating your containers?

How do you make the app inside a container accessible?

Scaling out with containers

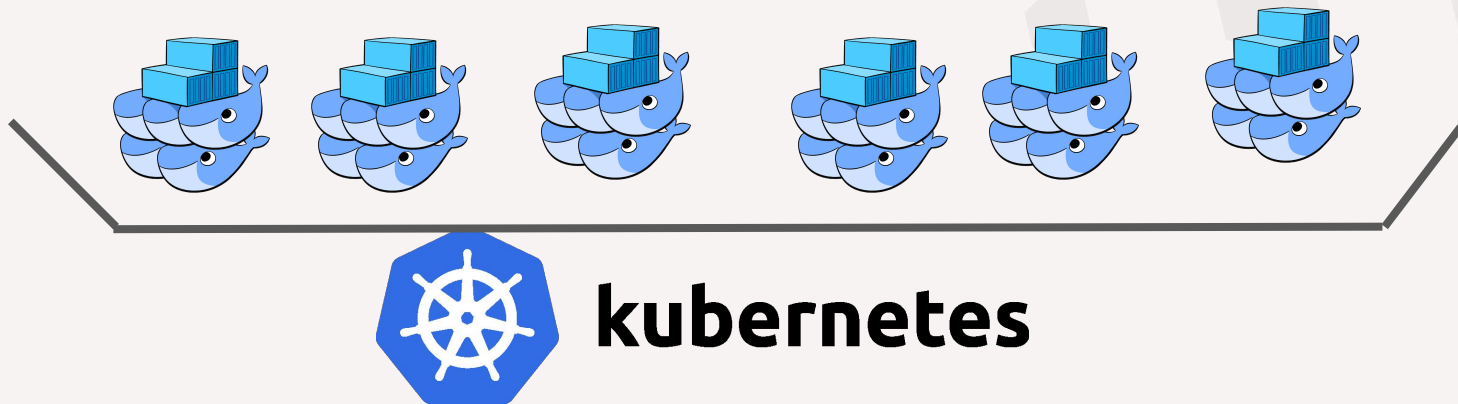
How do you add scalability and redundancy to your containers?

How do you handle updating your containers?

How do you make the app inside a container accessible?

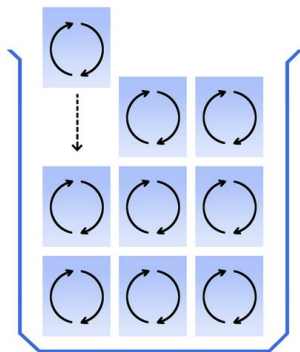
How do you manage multiple apps and containers?

Solutions



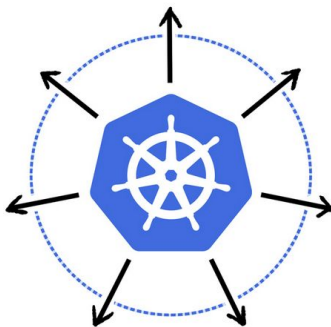
Kubernetes

TO DISTRIBUTE
CONTAINERS IN A **LOGICAL**
AND **EFFICIENT** WAY.



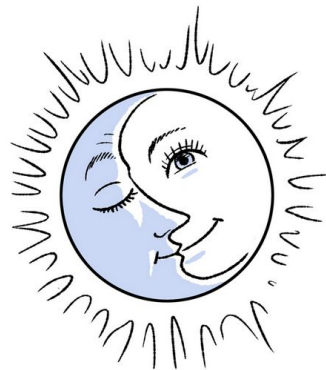
TRANSLATION:
MAXIMIZE CAPACITY

TO **SCALE UP** (OR DOWN)
FAST WITH THE OPS YOU
ALREADY HAVE.



TRANSLATION:
ADAPT TO DEMAND

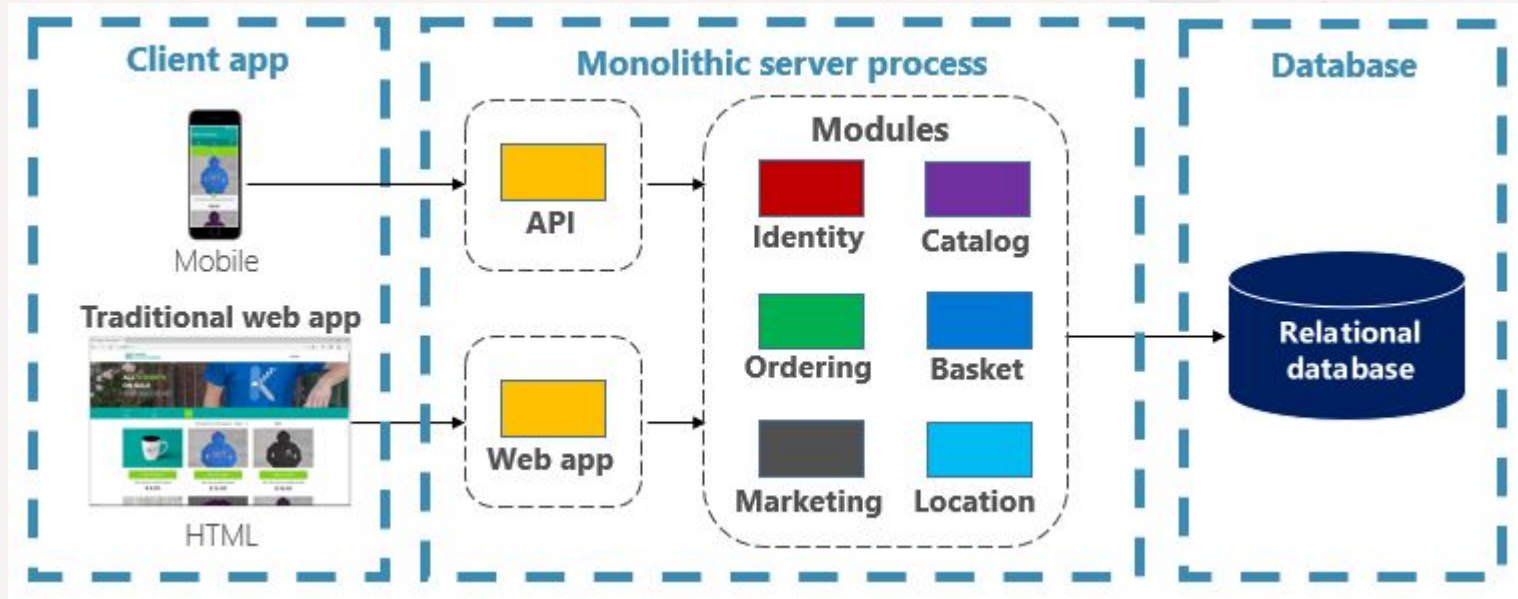
TO KEEP PROCESSES
CONTINUOUSLY **RUNNING**
AND **HEALTHY**.



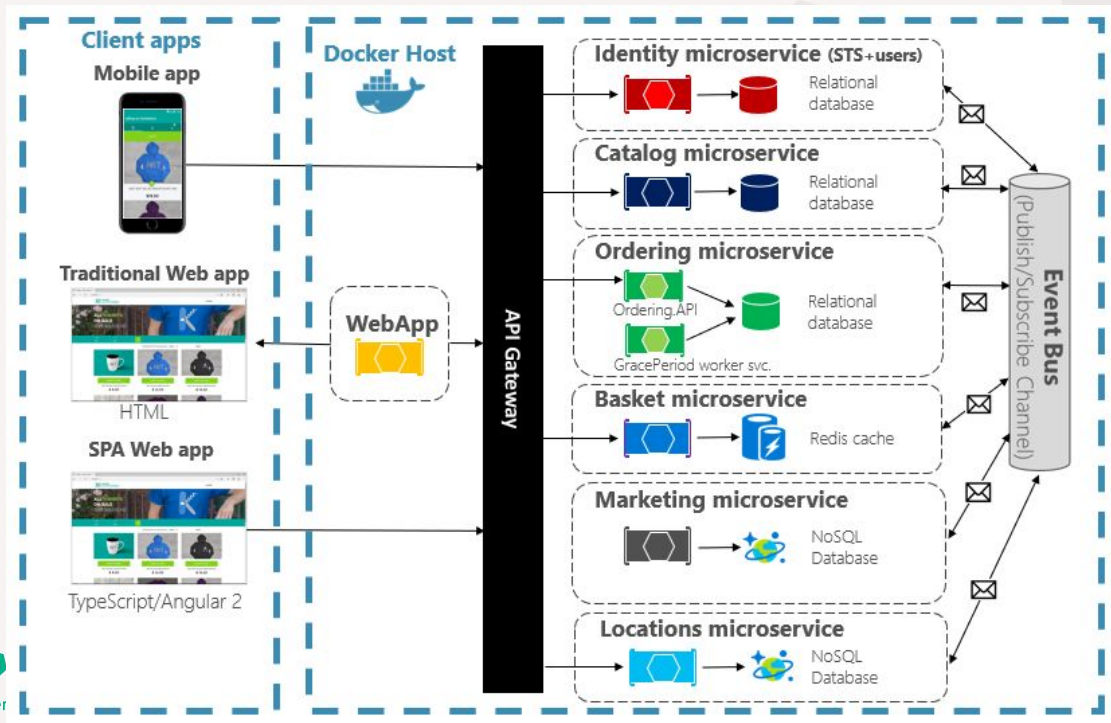
TRANSLATION:
DON'T GO DARK

Cloud native applications

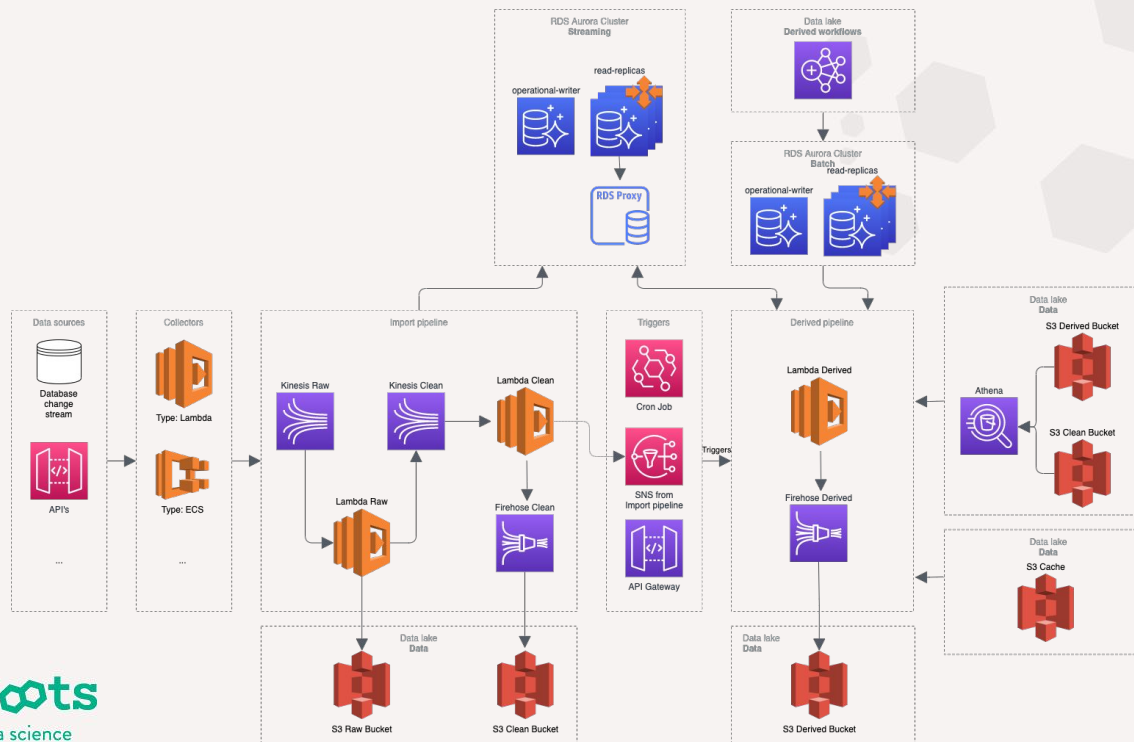
Example of Cloud Native Architecture Before



Example of Cloud Native Architecture After

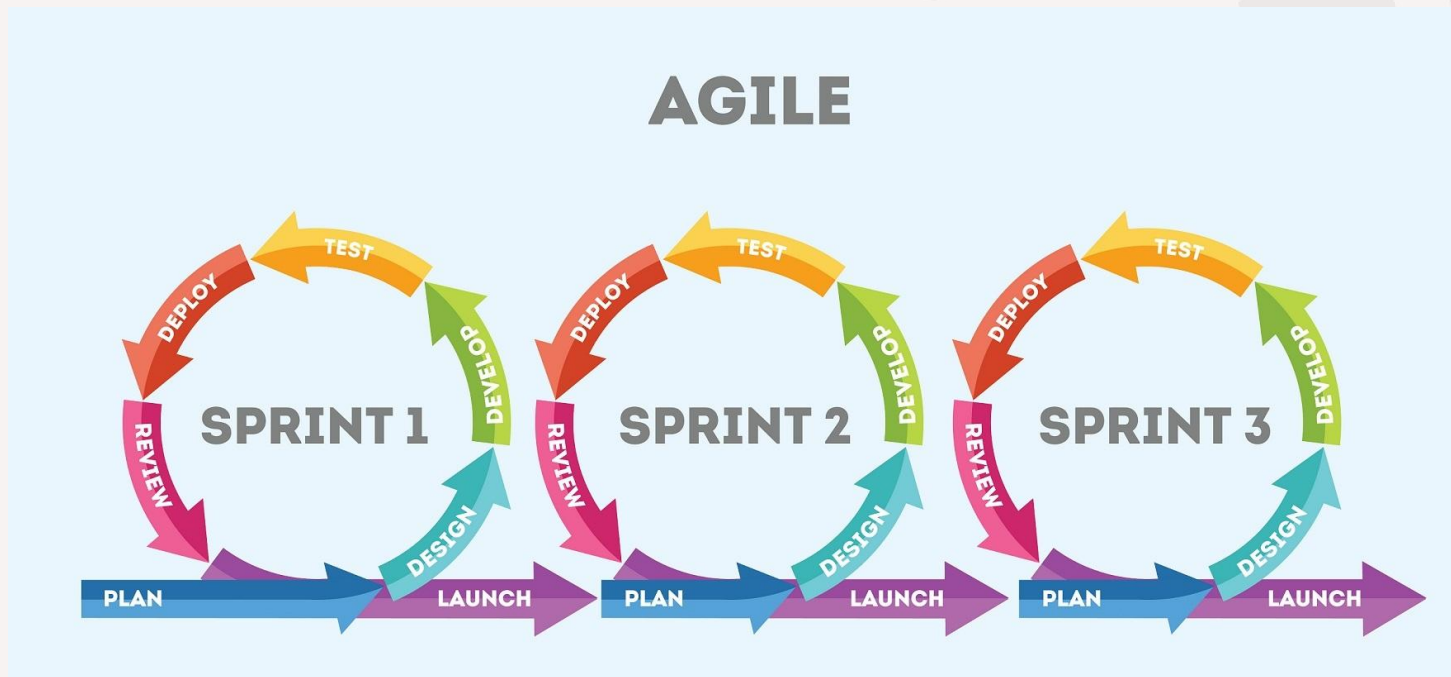


Example of Cloud Native Architecture



Agile, DevOps & CI/CD

What is Agile?





Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Through this work we have come to value:

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

What is DevOps?

*“A set of **practices** intended to reduce the time between committing a **change** to a system and the change being placed into normal production, while ensuring **high quality**”*

DevOps is a way to bring Agile into practice

What is DevOps?

Wall of Confusion



DEVELOPMENT

- Deliver new features
- Product oriented
- Innovation



OPERATIONS

- Guarantee stability
- Service oriented
- Rationalization

What is DevOps?

Software delivery performance metric	Elite	High	Medium	Low
 Deployment frequency For the primary application or service you work on, how often does your organization deploy code to production or release it to end users?	On-demand (multiple deploys per day)	Between once per week and once per month	Between once per month and once every 6 months	Fewer than once per six months
 Lead time for changes For the primary application or service you work on, what is your lead time for changes (i.e., how long does it take to go from code committed to code successfully running in production)?	Less than one hour	Between one day and one week	Between one month and six months	More than six months
 Time to restore service For the primary application or service you work on, how long does it generally take to restore service when a service incident or a defect that impacts users occurs (e.g., unplanned outage or service impairment)?	Less than one hour	Less than one day	Between one day and one week	More than six months
 Change failure rate For the primary application or service you work on, what percentage of changes to production or released to users result in degraded service (e.g., lead to service impairment or service outage) and subsequently require remediation (e.g., require a hotfix, rollback, fix forward, patch)?	0%-15%	16%-30%	16%-30%	16%-30%



DevOps is not about the tools

DevOps engineer?

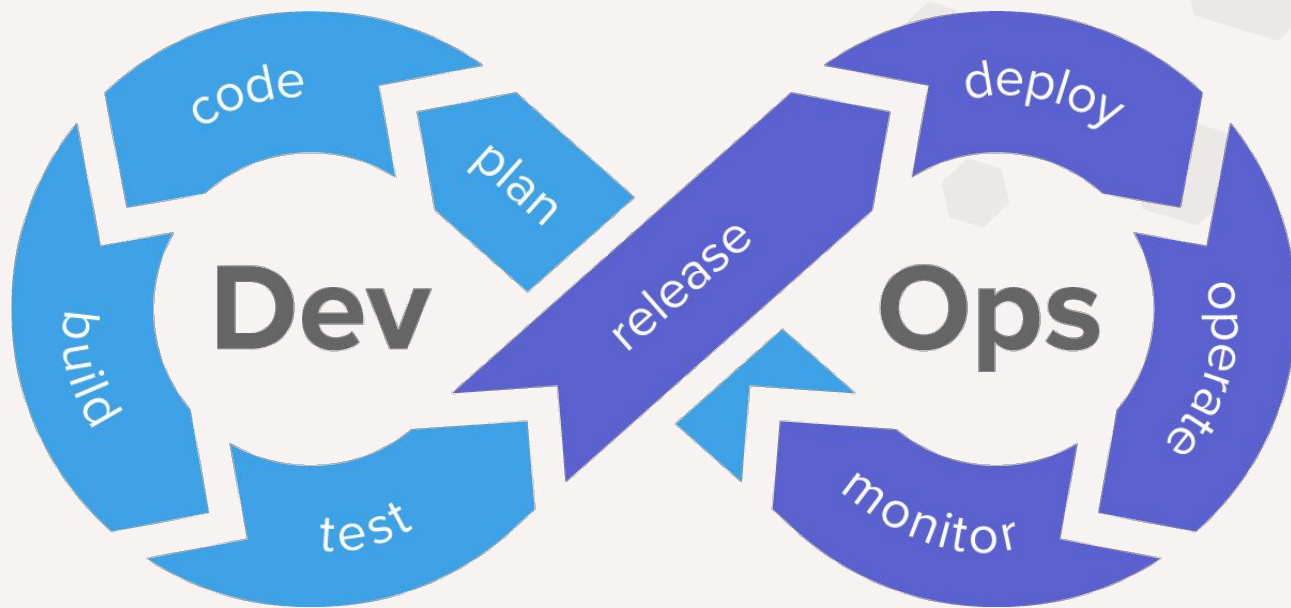
DevOps engineer

Role : **Enabler of Devops culture**

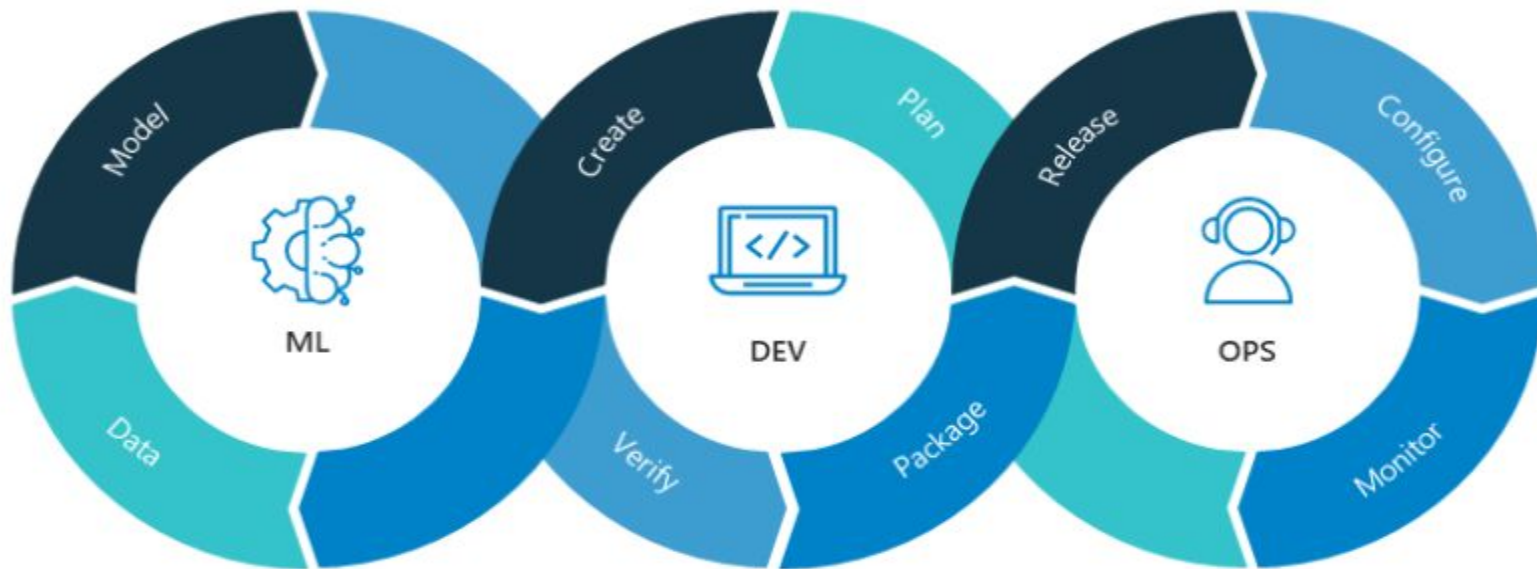
- Helps building CI/CD pipelines
- Setup monitoring, logging and alerting
- Ensures that tests and security is following high quality standards
- Improves infrastructure reliability (scale, self-resilience, etc)

Many companies lack the DevOps culture, thus confusing DevOps engineer with Infrastructure or Cloud engineer

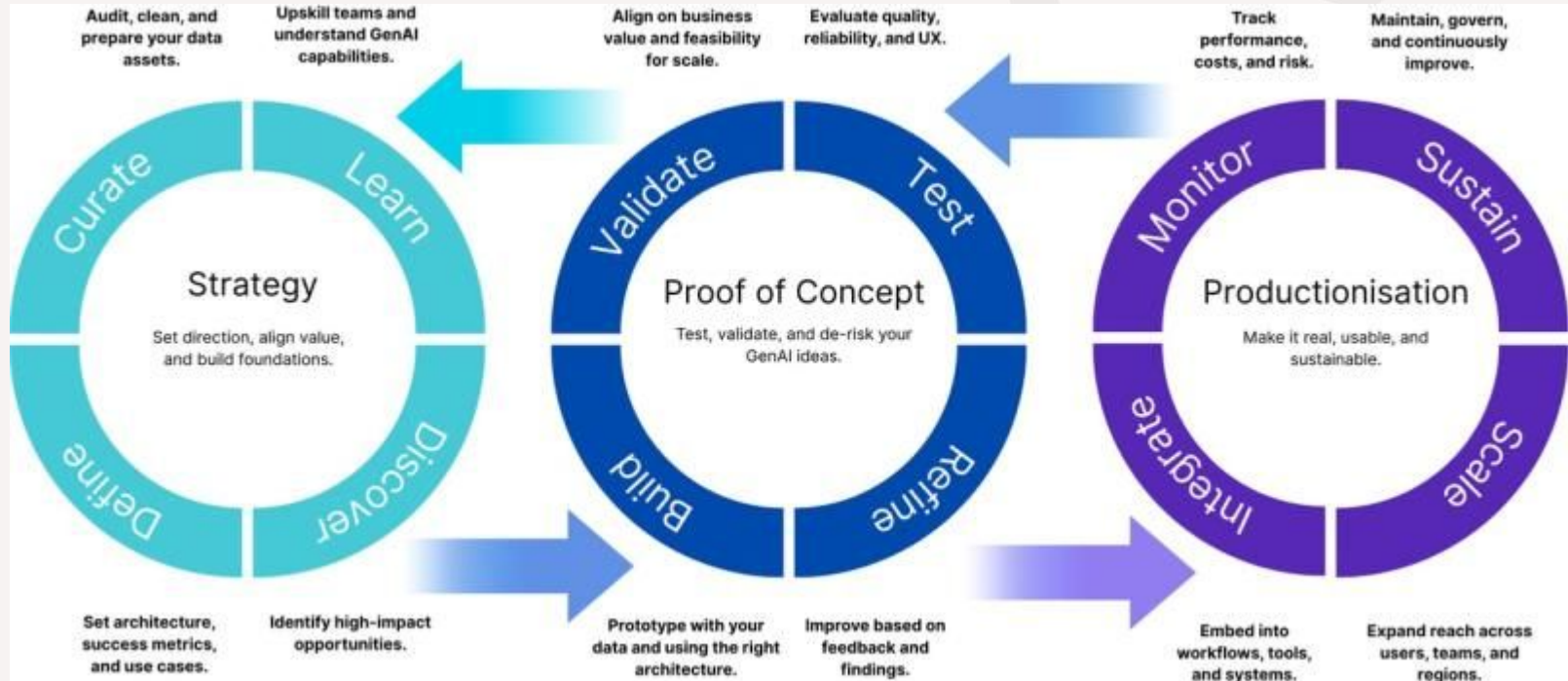
DevOps engineer



MLOps?



GenAIOps?



CI/CD

Tooling enablers : CI/CD

- CI = Continuous Integration
- CD
 - Continuous Delivery (phase 1)
 - Continuous Deployment (Phase 2)

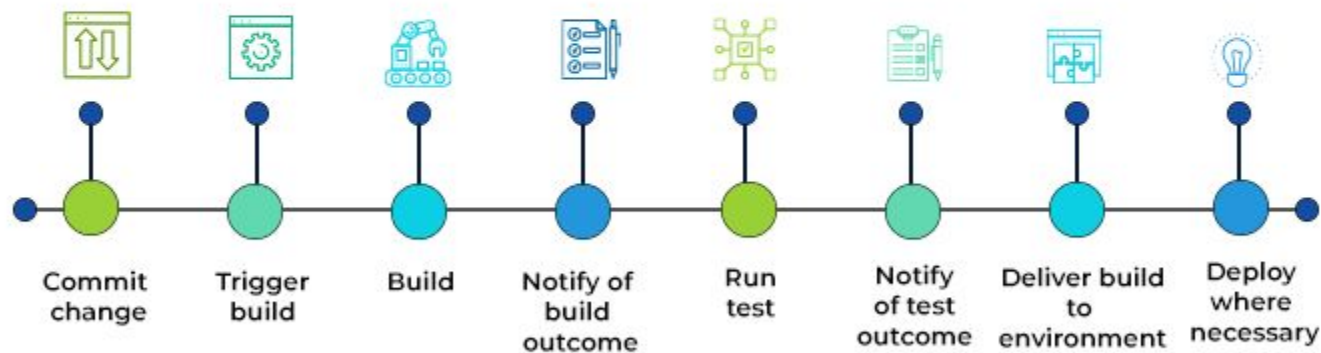
CI/CD in practice



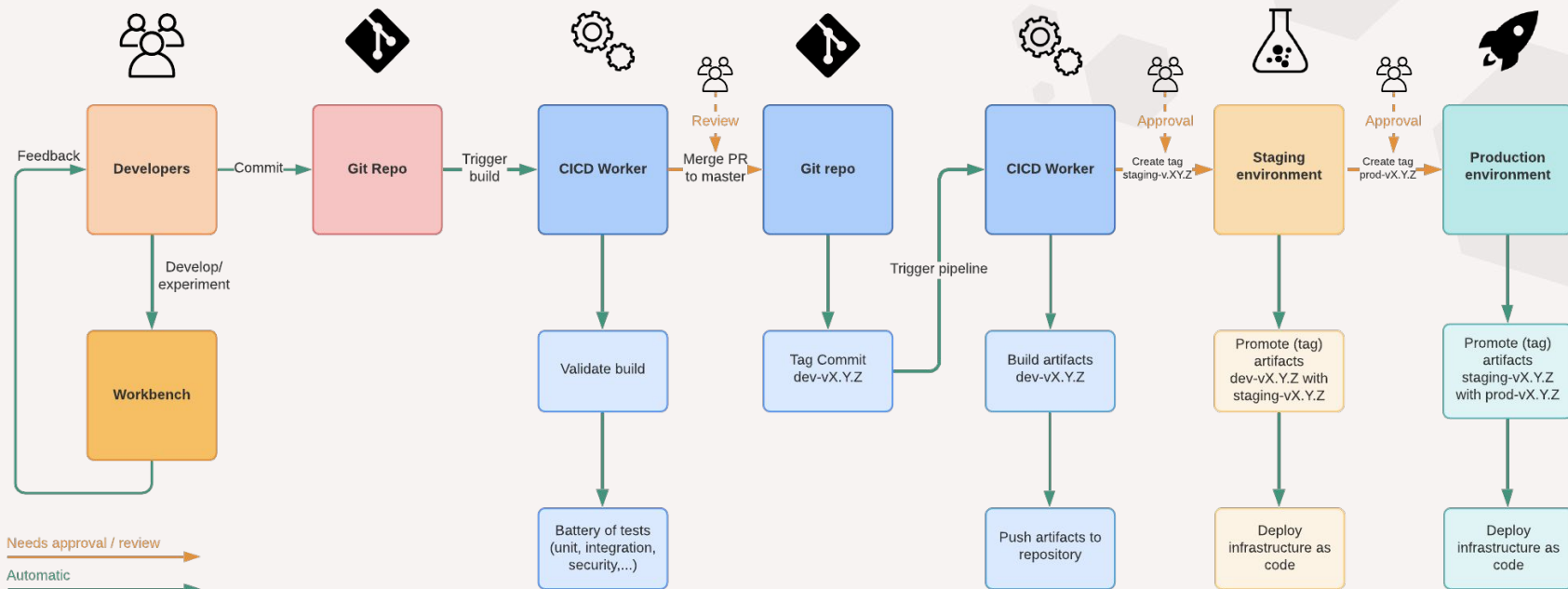
CI/CD in practice



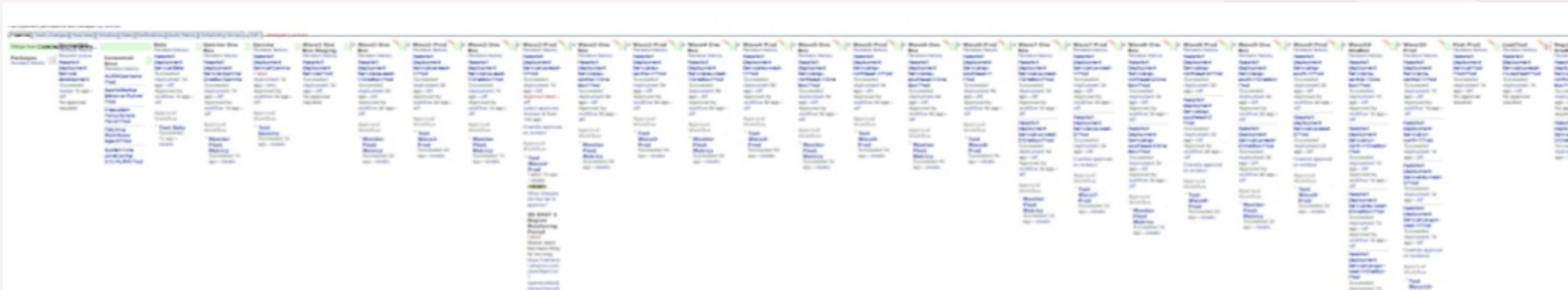
CI/CD PIPELINE



Tooling enablers : MLOps CI/CD example

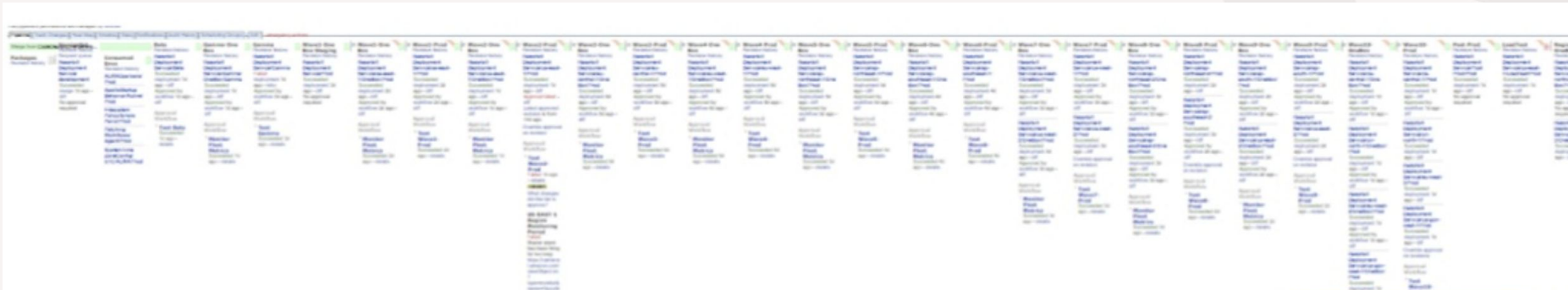


Tooling enablers : AWS CI/CD example



<https://youtu.be/l61KKO1rAQ8?t=1730>

Tooling enablers : AWS CI/CD example



Takeaways:

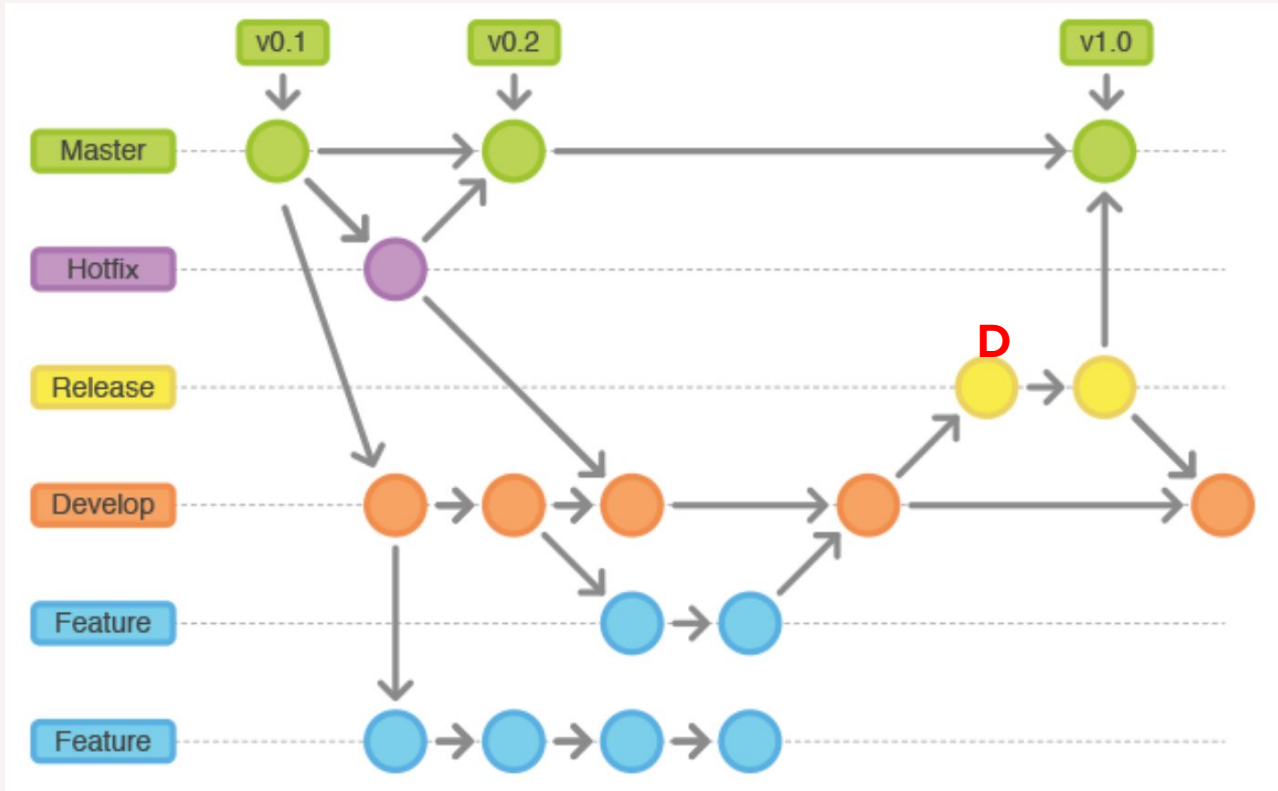
- Manual approvals
- Continuous testing
- Ship during business hours

CI/CD: takeaways

- It's about **releasing software**
- Consistent, reproducible, standardized releases
- All deployments to prod should come from a CI/CD pipeline
- CI/CD is triggered by git events (push/merge):
 - Can go back in time, revert mistakes
 - PR reviews
 - Tagging, branching,

Git Branching model

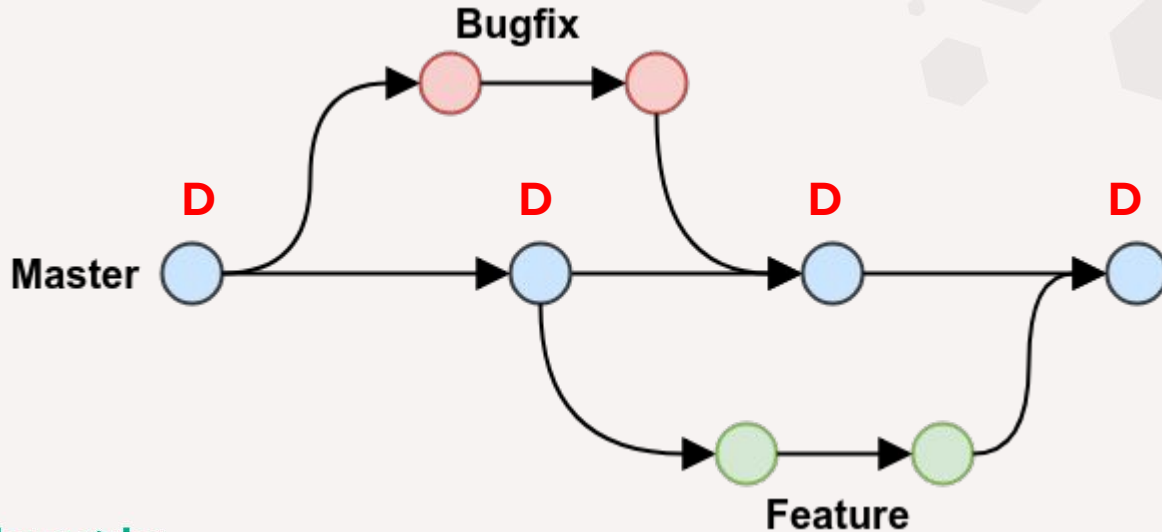
Tooling enablers : Git workflows



Built for “stability”

Tooling enablers : Git workflows

Trunk based development flow



Built for agility

Security 101



**Handle
secrets with
care**

**Least
privileged
access**

Zero trust

**Treat data
responsibly**

Handle secrets with care

NEVER store credentials in plain sight / code

NEVER send secrets via unprotected channels

Instead:

- Use key vault / secrets manager
- Use “one time password share” tool (<https://password.link/en>)

Handle secrets with care

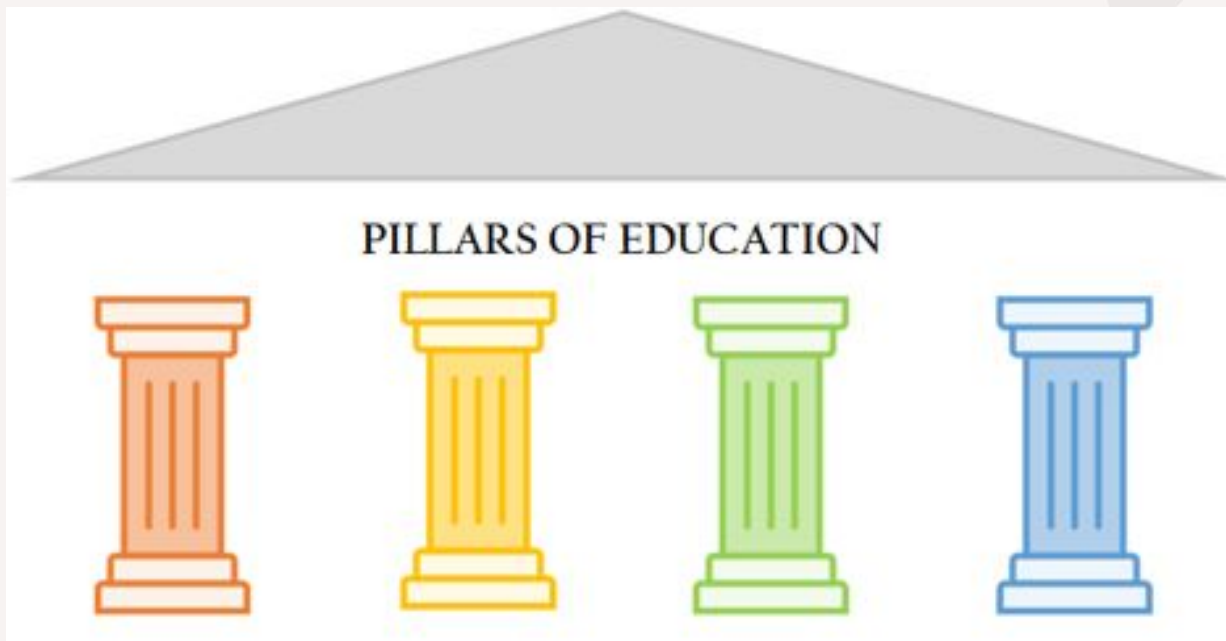
Demo Azure key vault

Handle secrets with care

 secrets.py ✕

Users > quinten.bruynseraede >  secrets.py > ...

```
1 import os
1
2 password = os.getenv("PASSWORD")
3 # use password here
```



**Handle
secrets with
care**

**Least
privileged
access**

Zero trust

**Treat data
responsibly**

Least privileged access

Least privileged access

ALICE



APP



BOB

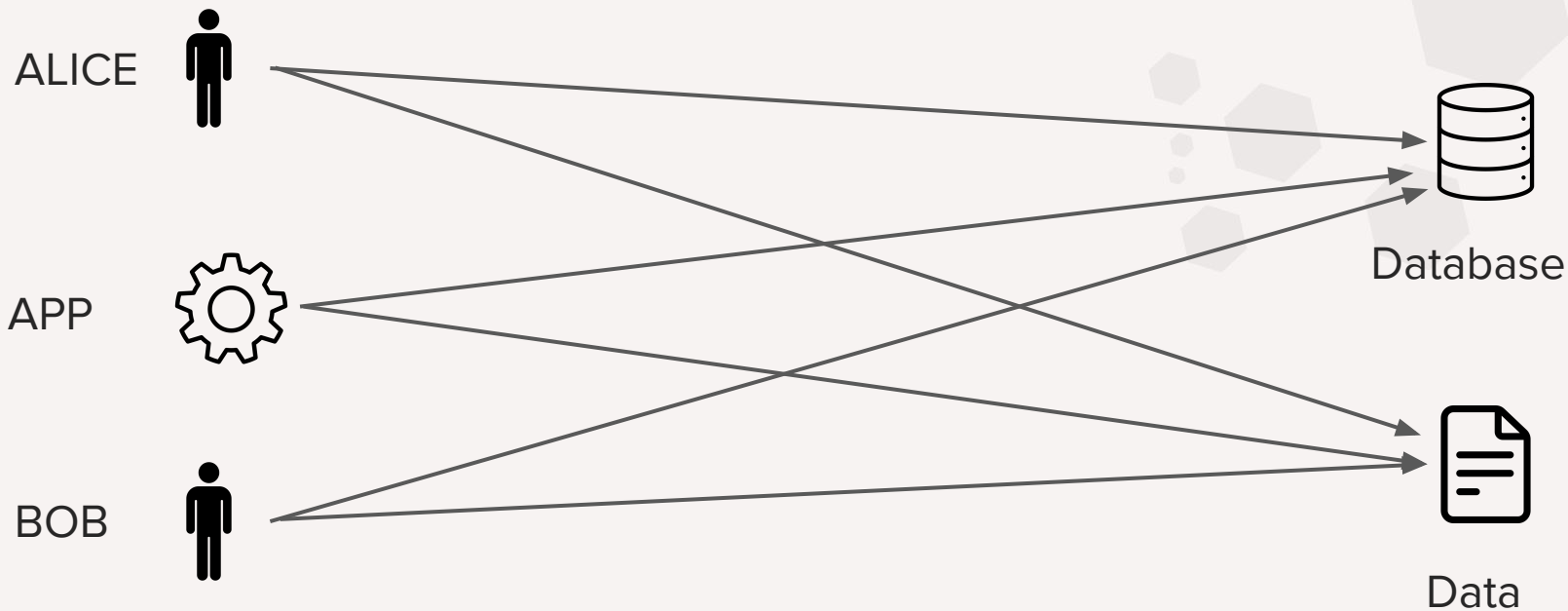


Database

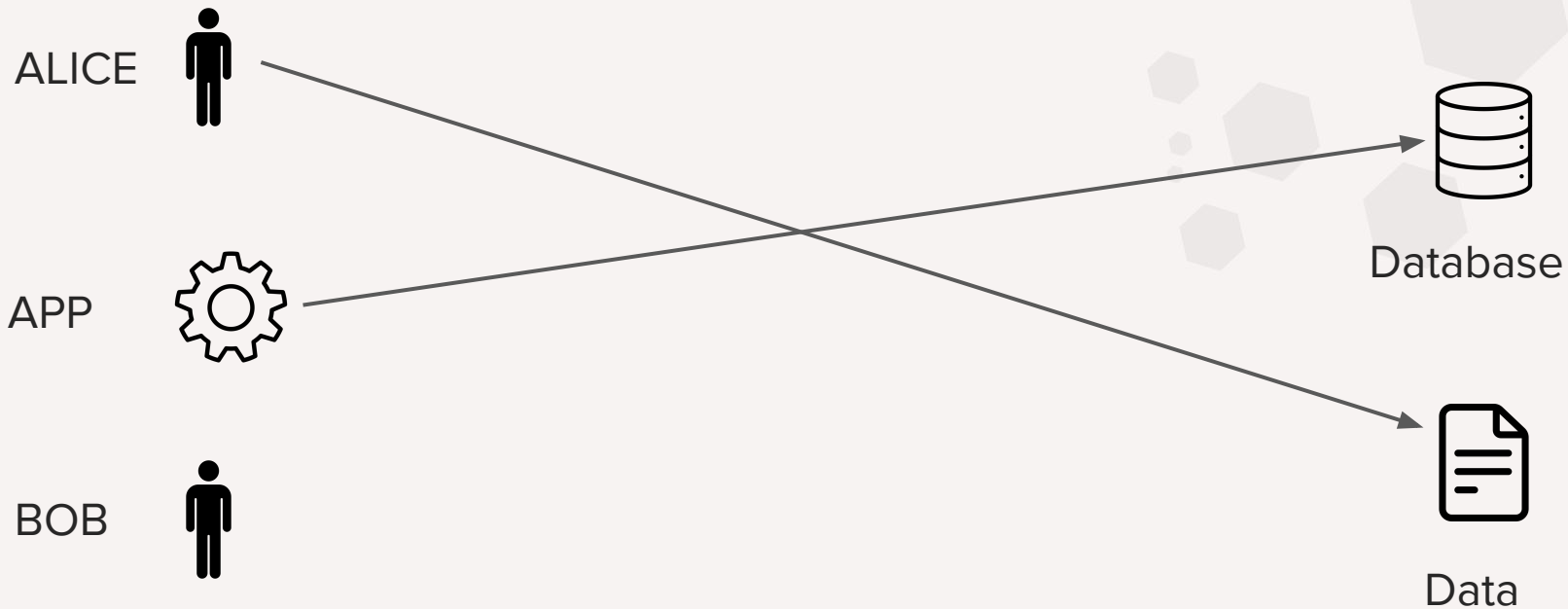


Data

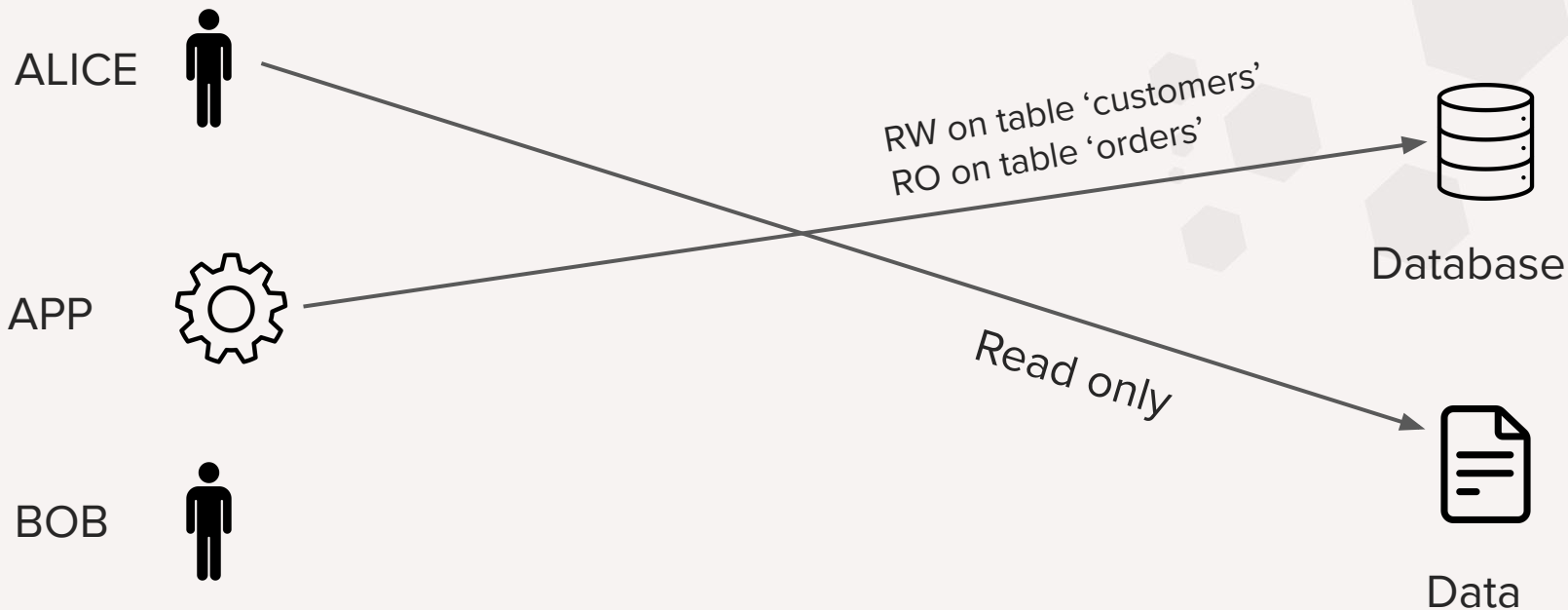
Least privileged access

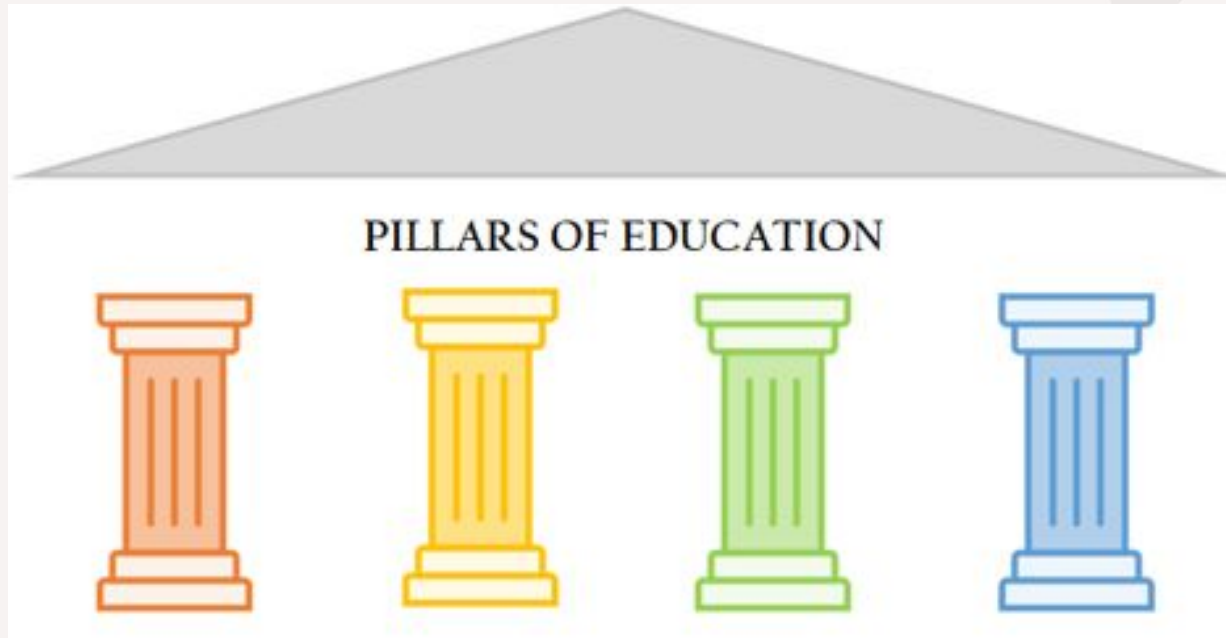


Least privileged access



Least privileged access





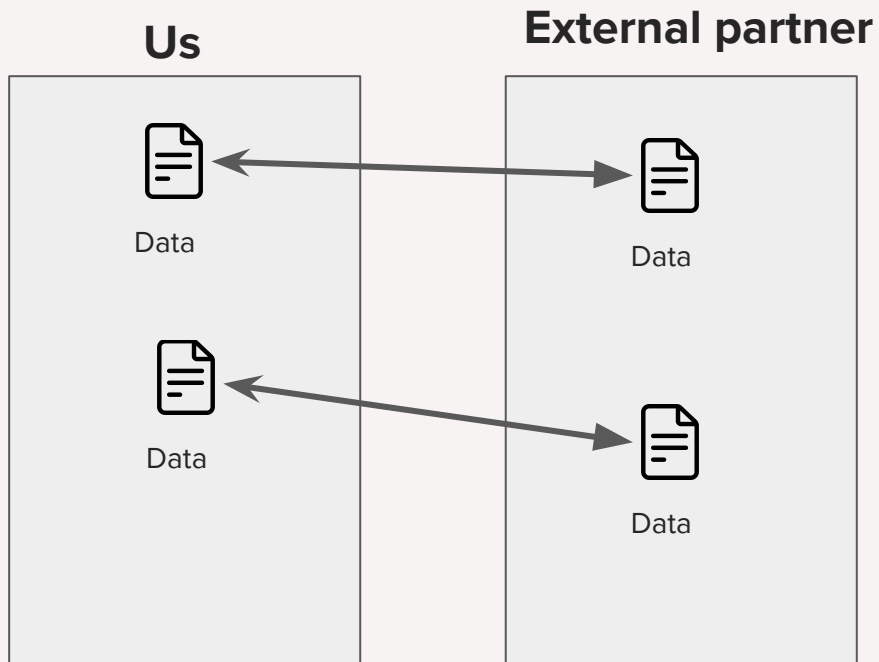
**Handle
secrets with
care**

**Least
privileged
access**

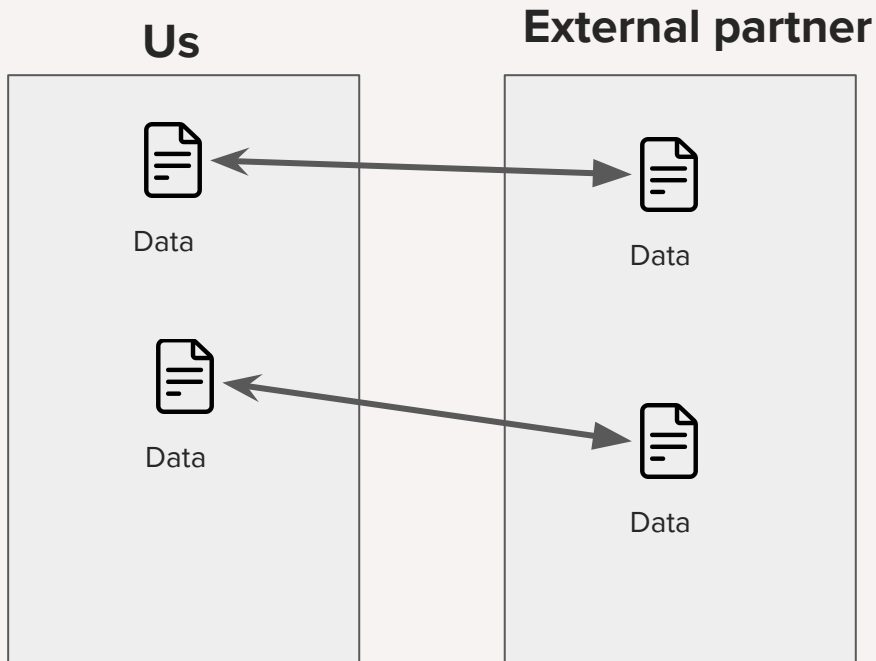
Zero trust

**Treat data
responsibly**

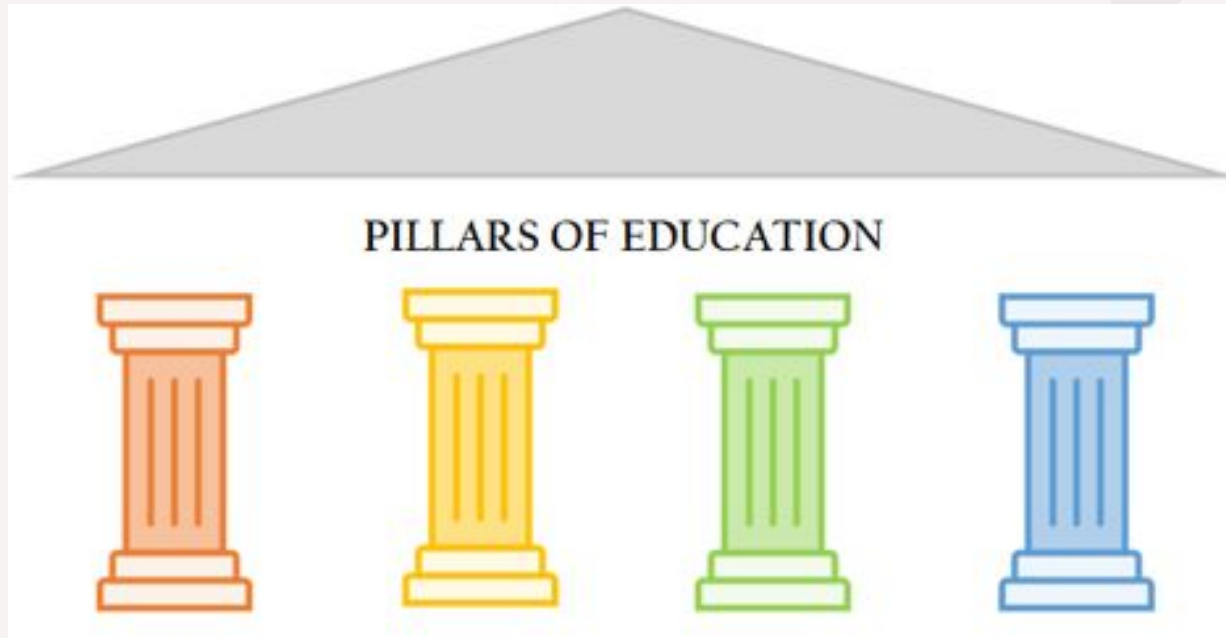
Zero trust



Zero trust



1. Continuous verification
2. Least privilege access
3. Assume breach



**Handle
secrets with
care**

**Least
privileged
access**

Zero trust

**Treat data
responsibly**

Treat data responsibly

1. No public/anonymous access unless needed

Treat data responsibly

1. No public/anonymous access unless needed
2. No customer data on your laptop

Treat data responsibly

1. No public/anonymous access unless needed
2. No customer data on your laptop
3. Notify about GDPR issues

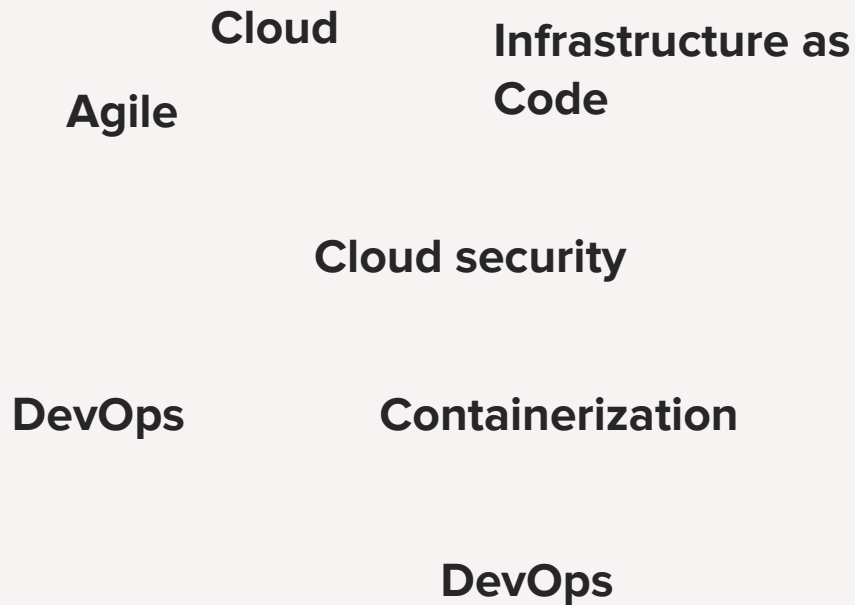
Treat data responsibly

1. No public/anonymous access unless needed
2. No customer data on your laptop
3. Notify about GDPR issues
4. When possible, work with anonymised data

(In case of doubts, ask your manager / tech lead)

Summary

Principles and tools



Questions ?